

Instruções de instalação e execução

Introdução

Este manual de instruções é um guia para a instalação e configuração das bibliotecas Flask, Jwt (Json Web Tokens) e psycopg2-binary num ambiente isolado Python, para o correto funcionamento da nossa aplicação. Usámos também bcrypt para proteger as passwords adicionadas à base de dados, pelo que procedemos à sua instalação.

Flask permite a criação de API's de forma simples, jwt permite criar e verificar tokens JWT (JSON Web Tokens), sendo útil também para a autenticação e autorização em API's e psycopg2-binary é uma biblioteca que permite a conexão entre a API e o PostgreSQL.

Pré-requisitos

Precisamos de ter instalado tanto o python como o pip, que é um gerenciador de pacotes do Python. Pode-se verificar se já estão instalados executando o seguinte comando no terminal:

```
python --version
```

```
pip --version
```

Para Linux e MacOS:

Passo 1. Criar o ambiente virtual isolado onde iremos instalar as bibliotecas

```
python3 -m venv projectSGD
```

Passo 2. Ativar o ambiente criado

```
source proj_sgd_env/bin/activate
```

Passo 3. Instalar os módulos necessários nesse ambiente

```
pip3 install flask
```

```
pip3 install jwt
```

```
pip3 install psycopg2-binary
```

```
pip3 install bcrypt
```

Passo 4. Executar a aplicação Flask

```
python3 demo-api-sgd.py
```

Para Windows:

Passo 1. Criar o ambiente isolado onde iremos instalar as bibliotecas

```
python -m venv projectSGD
```

Passo 2. Ativar o ambiente virtual

```
projectSGD/bin/activate
```

Passo 3. Importar os módulos necessários

```
pip install flask
```

```
pip install jwt
```

```
pip install psycpg2-binary
```

```
pip install bcrypt
```

Passo 4. Correr a aplicação flask

```
python demo-api-sgd.py
```

No caso do MacOS, estava a dar erro no início ao executar o código, pois aparecia que o psycpg2-binary não estava instalado, sendo que já havia procedido à sua instalação. A resolução foi ir desinstalando e voltando a instalar, atualizando também o pip e verificando a sua versão, até finalmente ser possível a sua utilização sem erros.

No caso do Windows, os comandos anteriores não foram o suficiente e o computador não encontrava o caminho do psycpg2-binary ao executar o código, então o que funcionou foi desinstalar o python, voltar a instalá-lo e fazer estas etapas anteriores de novo.

Caso estes erros surjam, pode-se recorrer às ideias anteriores que funcionaram no nosso caso (sem muitas certezas de funcionar para outras pessoas, pois cada computador apresenta um erro diferente).

Todos estes passos garantem que as dependências não entrem em conflito com outras versões de bibliotecas no sistema, proporcionando um ambiente controlado e seguro para o desenvolvimento de projeto.

Depois de termos este ambiente com as bibliotecas instaladas, passamos a explicar a parte correspondente à nossa base de dados, ao postman e ao nosso código.

Base de dados

Para este projeto, utilizámos o DBeaver para criar a nossa base de dados e retirámos o script para criar as tabelas do ONDA onde foi criado o ER. Adicionámos um server chamado “projeto” e acedemos a este através das credenciais de um novo user por nós criado que tem o mesmo nome que o server. A base de dados criada tem o nome de “projectSGD” e por isso colocámos essas informações na função db_connection no ficheiro “demo-api-sgd.py”. O código de criação de tabelas encontra-se no ficheiro “createTables.sql” dentro da pasta “postgresql”. As procedures criadas no DBeaver e os inserts iniciais nas tabelas estão inseridos no ficheiro “procedures_inserts.sql” na mesma pasta correspondente à criação de tabelas anteriormente mencionada.

Postman

Antes de utilizar o Postman, temos de nos assegurar de que executamos a aplicação Flask, ou seja, que executámos o ficheiro python “demo-api-sgd.py”, caso contrário este não poderá gerar as respostas pretendidas por não ter acesso às funções criadas.

Para este projeto foram criados dez endpoints, 4 com a operação ‘POST’, 1 com ‘PUT’ e 5 com ‘GET’. Todas as operações que não são do tipo ‘GET’ necessitam de receber no “Body” em “raw” um payload, um dicionário com informações que irão ser utilizadas para a função correspondente presente no código em python.

Para além de tudo isto, em alguns procedimentos era necessário que o utilizador tivesse uma autorização específica para poder aceder a eles e por isso tivemos de alterar no parâmetro “Authorization”. Esse parâmetro inicialmente estava definido como “Inherit auth from parent” mas foi modificado para “Bearer token”. Desta forma, um utilizador de um determinado tipo autorizado, ao realizar o login, recebe um token que coloca no parâmetro “Authorization” e que o permite então ter acesso a um determinado endpoint restrito.

O funcionamento de um endpoint apenas é possível através de um URL que, no nosso caso, começa sempre com “<http://localhost:8080/>” e depois de “/” é adicionada uma determinada route (dependendo do endpoint em questão e da ação que queremos que este execute) que está associada a uma certa função no nosso código em python.

Resumindo, de forma a utilizar um endpoint, temos então, consoante o seu tipo, de passar ou não um dicionário com informação (payload) e de adicionar ou não um token de autorização que é devolvido no login apenas a um tipo de utilizador específico autorizado. Por fim, temos apenas de pressionar “Send” para enviar o request.

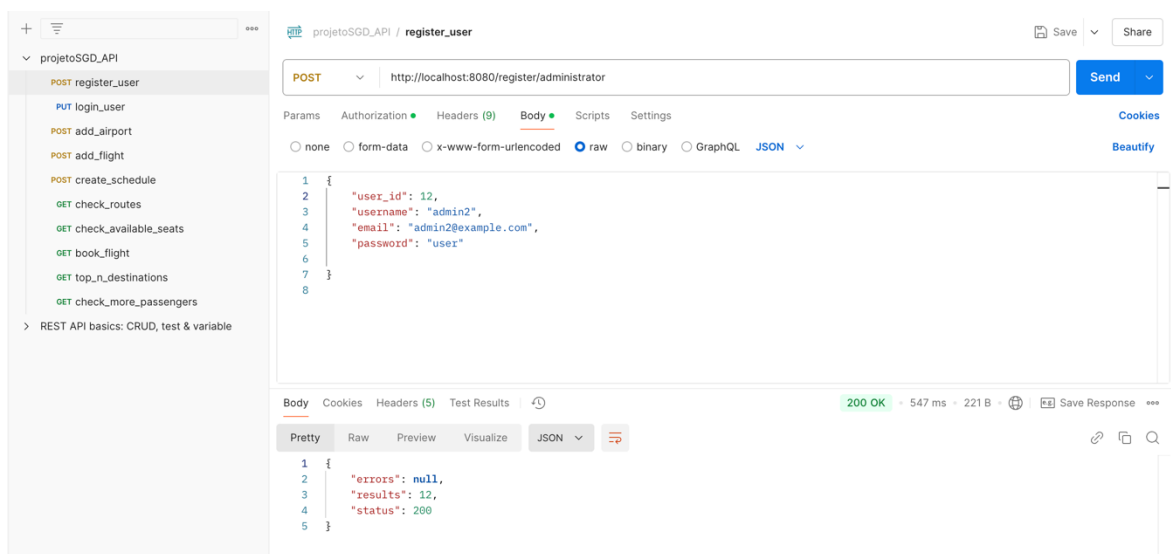


Figura 1 – Exemplo de endpoint no Postman

Código

Para o nosso código não dar erro a executar no virtual environment, temos de abrir um terminal na pasta “app” que contém esse mesmo ambiente e a pasta com o ficheiro log, usando os comandos para correr a aplicação flask nesse terminal.

Conclusão

Desta forma, é possível entender mais facilmente o funcionamento de todos estes elementos em conjunto e a maneira como estão a ser utilizados e manipulados no nosso projeto.